

Proyecto 2a. Evaluación

COMPUTO PARALELO Y DISTRIBUIDO

Presentado por:

HECTOR RODRIGUEZ LOYA
LUIS ARTURO HERNANDEZ CASTILLO
INGRID YULIANA PEREZ RODRIGUEZ
JAZMIN CRUZ GONZALES

Docente:

ING. JOSE SAUL DE LIRA MIRAMONTES



UNIVERSIDAD AUTÓNOMA DE
CHIHUAHUA

Facultad de Ingeniería

May - 2026

Índice de Contenido

1. Descripción General del Proyecto	3
2. Arquitectura del Sistema	3
3. Flujo General del Pipeline	3
4. Scheduler Asíncrono	4
5. Obtención de Datos de Vuelos	4
5.1. Llamada Principal	4
5.2. Distribución de Información	4
5.3. Conversión de Coordenadas	5
5.4. Ventajas del Paralelismo	5
6. Recolección y Escritura de Resultados	5
7. Consulta de Clima y Predicción de Retrasos	5
7.1. Datos Obtenidos	5
7.2. Procesamiento del Modelo	5
7.3. Escritura de Resultados	6
7.4. Posibles Mejoras	6
8. Dashboard Web	6
8.1. Funcionalidades del Dashboard	6
9. Tecnologías Utilizadas	6
10. Ventajas de la Arquitectura	7
11. Posibles Mejoras Futuras	7
12. Conclusión	7

1. Descripción General del Proyecto

El proyecto consiste en el desarrollo de un sistema distribuido orientado al monitoreo de vuelos y predicción de retrasos utilizando procesamiento paralelo, programación asíncrona y visualización en tiempo real.

La arquitectura combina:

- **MPI** para distribuir procesamiento entre múltiples ranks.
- **Asyncio** para realizar peticiones concurrentes a APIs.
- Un **scheduler asíncrono** encargado de ejecutar periódicamente el pipeline.
- Un **dashboard web** para visualizar los resultados en tiempo real.

El sistema trabaja continuamente obteniendo información de vuelos sobre México, procesando coordenadas geográficas, consultando datos meteorológicos y generando predicciones de retraso para aeropuertos.

2. Arquitectura del Sistema

Componente	Función
Scheduler	Ejecuta el pipeline cada cierto intervalo de tiempo
Rank 0	Obtiene datos globales y centraliza resultados
Ranks Workers	Procesan información distribuida
API de vuelos	Entrega información de aeronaves sobre México
API de clima	Entrega condiciones meteorológicas por aeropuerto
Modelo predictivo	Predice retrasos de vuelos
Archivos JSON/CSV	Almacenan resultados procesados
Dashboard HTML	Visualiza información en tiempo real

3. Flujo General del Pipeline

El pipeline se ejecuta periódicamente mediante un scheduler asíncrono.

Las etapas principales son:

- Obtención de datos de vuelos.
- Distribución de información entre ranks.
- Procesamiento paralelo de coordenadas.
- Recolección y almacenamiento de resultados.
- Consulta de clima y predicción de retrasos.
- Actualización del dashboard.

4. Scheduler Asíncrono

El scheduler es responsable de ejecutar el pipeline cada cierto intervalo de tiempo sin bloquear la ejecución principal.

Se utiliza programación asíncrona para:

- Mantener el sistema activo continuamente.
- Ejecutar múltiples peticiones concurrentes.
- Reducir tiempos de espera por operaciones de red.

Ejemplo simplificado:

```
async def main():
    scheduler = Scheduler(
        interval=5,
        callback=pipeline
    )

    await scheduler.start()
```

Ventajas:

- Mejor aprovechamiento del tiempo de espera de APIs.
- Mayor capacidad de escalabilidad.
- Menor bloqueo del sistema.

5. Obtención de Datos de Vuelos

5.1. Llamada Principal

El rank 0 es el encargado de realizar una única petición a la API de vuelos.

La API devuelve información como:

- Latitud
- Longitud
- Altitud
- Dirección
- Velocidad
- Identificador del vuelo

5.2. Distribución de Información

Después de obtener los datos:

Rank 0 divide la información en fragmentos. Cada fragmento se envía a un rank diferente utilizando MPI.

Esto permite:

Distribuir la carga de procesamiento. Reducir tiempo total de ejecución.

Cada rank recibe una parte de los vuelos y procesa únicamente sus datos locales.

5.3. Conversión de Coordenadas

Los datos originales utilizan:

Latitud Longitud

Estos valores se transforman a coordenadas cartesianas:

x y

Esto facilita:

Graficación en el dashboard. Cálculos espaciales. Representación visual de trayectorias.

5.4. Ventajas del Paralelismo

Procesamiento más rápido. Mejor uso del CPU. Escalabilidad para grandes cantidades de vuelos.

6. Recolección y Escritura de Resultados

Una vez terminado el procesamiento:

- Cada rank devuelve sus resultados al rank 0.
- Rank 0 junta toda la información.

Después:

- Los datos se escriben a un archivo CSV.
- El dashboard puede leerlos posteriormente.

7. Consulta de Clima y Predicción de Retrasos

Mientras el scheduler espera la siguiente ejecución:

Cada rank realiza consultas concurrentes a una API de clima. Se consulta información meteorológica de aeropuertos de México.

7.1. Datos Obtenidos

La API puede devolver:

Temperatura Humedad Visibilidad Velocidad del viento Condiciones climáticas

7.2. Procesamiento del Modelo

Los datos climáticos se introducen a un modelo predictivo encargado de estimar retrasos de vuelos.

Entradas posibles:

- Clima
- Velocidad del viento
- Visibilidad
- Hora del día
- Aeropuerto

Salida:

- Probabilidad de retraso
- Tiempo estimado de retraso

7.3. Escritura de Resultados

Las predicciones se almacenan en archivos para ser consumidos por el dashboard.

7.4. Posibles Mejoras

Actualmente:

Esta tarea se ejecuta únicamente durante la primera vuelta del scheduler.

A futuro:

Podría ejecutarse en otro scheduler independiente. Ambos pipelines podrían correr paralelamente. Se podría separar: monitoreo de vuelos predicción meteorológica

8. Dashboard Web

El frontend consiste en un dashboard HTML que:

- Lee continuamente los archivos generados.
- Actualiza información en tiempo real.

La visualización puede incluir:

- Posición de aviones.
- Trayectorias.
- Estado de aeropuertos.
- Predicciones de retrasos.

8.1. Funcionalidades del Dashboard

- Actualización automática.
- Visualización en tiempo real.
- Integración de múltiples fuentes de datos.
- Monitoreo continuo.

9. Tecnologías Utilizadas

Tecnología	Uso
Python	Backend principal
asyncio	Concurrencia asíncrona
mpi4py	Procesamiento distribuido
NumPy	Manipulación de datos
JSON	Intercambio de información
HTML/CSS/JavaScript	Dashboard web
APIs REST	Obtención de datos externos

10. Ventajas de la Arquitectura

- Procesamiento distribuido eficiente.
- Escalabilidad horizontal.
- Actualización continua de información.
- Separación clara de responsabilidades.
- Integración sencilla con dashboards y modelos predictivos.

11. Posibles Mejoras Futuras

- Uso de WebSockets para actualización en tiempo real.
- Almacenamiento en bases de datos.
- Contenedores Docker.
- Despliegue en Kubernetes.
- Entrenamiento continuo del modelo predictivo.
- Uso de GPU para inferencia.
- Sistema de alertas automáticas.

12. Conclusión

El proyecto implementa una arquitectura moderna basada en concurrencia y procesamiento distribuido para el monitoreo de vuelos y predicción de retrasos.

La combinación de:

- MPI
- asyncio
- APIs externas
- modelos predictivos
- dashboards en tiempo real

permite construir un sistema escalable y eficiente capaz de manejar grandes volúmenes de información aeronáutica y meteorológica.

El diseño modular facilita futuras mejoras y permite separar tareas críticas en pipelines independientes para aumentar rendimiento y mantenibilidad.